



Docker — Complete Notes

Containers · Build, ship, and run apps anywhere

 Docker Containers

Tech Containerization

Isolation Namespaces + cgroups

Run Anywhere

A reader-friendly, all-in-one reference for Docker.

Table of Contents

1. [What is Docker](#)
2. [Containers vs Virtual Machines](#)
3. [Core Concepts & Architecture](#)
4. [Images & Layers](#)
5. [Dockerfile](#)
6. [Image Best Practices](#)
7. [Working with Containers](#)
8. [Data: Volumes & Bind Mounts](#)
9. [Networking](#)
10. [Registries \(Docker Hub & others\)](#)
11. [Docker Compose](#)
12. [Essential CLI Commands](#)
13. [Security Best Practices](#)
14. [Beyond Docker \(Orchestration\)](#)
15. [Quick Mental Model](#)

1. What is Docker

Docker is a platform to **build, package, and run applications in containers** — lightweight, isolated units that bundle your app with **everything it needs** (code, runtime, libraries, config). A container runs the same on a laptop, a CI runner, or a production server.

 **Mental model:** a container is a **running instance of an image**. The image is the immutable recipe/snapshot; the container is the live process. *"Build once, run anywhere."*

Why it matters: consistent environments (kills "works on my machine"), fast startup, dense resource use, and a clean unit for CI/CD and microservices.

2. ⚖️ Containers vs Virtual Machines

	Container	Virtual Machine
Isolates	A process (shares the host kernel)	A whole machine (its own OS kernel)
Size	MBs	GBs
Startup	Milliseconds–seconds	Seconds–minutes
Overhead	Minimal	Hypervisor + full guest OS
Isolation	OS-level (namespaces + cgroups)	Hardware-level (stronger)
Density	Many per host	Fewer per host

Containers **share the host kernel** — that's why they're small and fast, but isolation is weaker than a VM. Linux containers on Windows/macOS actually run inside a lightweight Linux VM.

3. 🧩 Core Concepts & Architecture

Concept	Description
Image	Read-only template (layers) used to create containers.
Container	A runnable instance of an image (an isolated process).
Dockerfile	Text recipe to build an image.
Registry	Store/distribute images (Docker Hub, ECR, GHCR...).
Volume	Persistent, Docker-managed storage.
Network	Virtual network connecting containers.
Docker daemon (<code>dockerd</code>)	Background service that builds/runs/manages objects.
Docker client (<code>docker</code>)	CLI that talks to the daemon (via REST API).
containerd / runc	Lower-level runtime that actually runs containers.

How it fits: the **client** sends commands → the **daemon** builds images, pulls from **registries**, and runs **containers** via **containerd/runc**, using Linux **namespaces** (isolation) and **cgroups** (resource limits).

4. 🧱 Images & Layers

- An image is a **stack of read-only layers**; each Dockerfile instruction (`RUN` , `COPY` , `ADD`) adds a layer.
- Layers are **cached and shared** — unchanged layers are reused across builds and images (fast builds, small pulls).

- A running container adds a thin **writable layer** on top (copy-on-write); deleting the container discards it.
- Images are identified by **name:tag** (e.g. `nginx:1.27`) and a content **digest** (`sha256:...`).
- **Base image** = the starting layer (`FROM ...`), e.g. `alpine` , `ubuntu` , `python:3.12-slim` , or `scratch` (empty).

💡 Order Dockerfile steps from **least- to most-frequently changing** so the cache survives — put dependency installs before copying source code.

5. Dockerfile

A Dockerfile is the build recipe. Common instructions:

Instruction	Purpose
FROM	Base image to start from
WORKDIR	Set the working directory
COPY / ADD	Copy files in (<code>ADD</code> also untars / fetches URLs)
RUN	Execute a command at build time (creates a layer)
ENV	Set environment variables
ARG	Build-time variable
EXPOSE	Document the port the app listens on
CMD	Default command at run time (overridable)
ENTRYPOINT	The fixed executable (args from <code>CMD</code> /CLI)
VOLUME	Declare a mount point
HEALTHCHECK	How Docker tests container health
USER	Run as a non-root user

```
# Multi-stage build: small, no build tools in the final image
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:1.27-alpine
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
HEALTHCHECK CMD wget -qO- http://localhost/ || exit 1
CMD ["nginx", "-g", "daemon off;"]
```

CMD vs ENTRYPOINT: `ENTRYPOINT` is the program; `CMD` supplies default args. Use `ENTRYPOINT ["app"] + CMD ["--flag"]` for a fixed binary with overridable arguments.

6. ✨ Image Best Practices

- **Small base images** — `alpine` / `-slim` / `distroless` ; less to ship and a smaller attack surface.
- **Multi-stage builds** — compile in a "build" stage, copy only artifacts into a lean final stage.
- `.dockerignore` — keep `node_modules` , `.git` , secrets out of the build context.
- **Few, ordered layers** — combine related `RUN` s; put stable steps first for cache hits.
- **Pin versions** — `python:3.12.4-slim` , not `latest` , for reproducible builds.
- **Run as non-root** (`USER`), set a `HEALTHCHECK` , and avoid baking secrets into layers (they persist in history).
- **One main process per container** — scale by running more containers, not by stuffing many services into one.

7. 📦 Working with Containers

```
docker run -d --name web -p 8080:80 nginx:1.27 # detached, map host:container
docker run -it ubuntu bash # interactive shell
docker ps # running containers (add -a for all)
docker logs -f web # follow logs
docker exec -it web sh # shell into a running container
docker stop web && docker start web
docker rm web # remove (must be stopped, or -f)
```

- `-d` detached · `-it` interactive TTY · `-p host:container` port map · `-e KEY=val` env · `--rm` auto-remove on exit · `--restart unless-stopped` restart policy · `--memory` / `--cpus` resource limits.
- **Container states:** `created` → `running` → `paused` → `stopped/exited` → `removed` .

8. 🗄️ Data: Volumes & Bind Mounts

Containers are **ephemeral** — the writable layer dies with the container. Persist data outside it:

Type	What	Use
Volume	Docker-managed storage (<code>/var/lib/docker/volumes</code>)	Preferred for persistent data (DB files). Portable, backup-able.
Bind mount	Maps a host path into the container	Local dev (live-edit source), host config files.
tmpfs	In-memory only	Sensitive/temp data that must not hit disk.

```

docker volume create dbdata
docker run -v dbdata:/var/lib/postgresql/data postgres # named volume
docker run -v "$(pwd)":/app node:20 # bind mount (dev)

```

Use **named volumes** for stateful services (databases) and **bind mounts** for live-reloading code during development.

9. Networking

Docker creates virtual networks so containers can talk to each other and the outside.

Driver	Behavior
bridge (default)	Private network on the host; containers get internal IPs. On a user-defined bridge, containers resolve each other by name (built-in DNS).
host	Container shares the host's network stack (no isolation, no port mapping).
none	No networking.
overlay	Multi-host network (Swarm / clustered services).
macvlan	Container gets its own MAC/IP on the physical LAN.

- **Port publishing:** `-p 8080:80` exposes container port 80 as host 8080. Without `-p`, ports are reachable only within the Docker network.
- Create a network and attach containers so they reach each other by service name:

```

docker network create appnet
docker run -d --name db --network appnet postgres
docker run -d --name api --network appnet -p 3000:3000 myapi # api reaches db as "db"

```

10. Registries (Docker Hub & others)

- A **registry** stores and distributes images; a **repository** holds tagged versions of one image.
- **Docker Hub** is the default public registry. Others: **Amazon ECR**, **GitHub GHCR**, **GitLab**, **Google Artifact Registry**, or a self-hosted registry.

```

docker login # authenticate
docker pull nginx:1.27 # download
docker tag myapp:1.0 user/myapp:1.0 # name for a repo
docker push user/myapp:1.0 # upload

```

Tag images meaningfully (semver + git SHA), and treat `latest` as "most recent build," **not** a stable pin for production.

13. 🛡️ Security Best Practices

- **Don't run as root** — add a `USER` ; consider rootless Docker.
- **Trusted, pinned base images** — official/verified, pinned by tag or digest; rebuild to get patches.
- **Scan images** for CVEs (`docker scout` , Trivy, Gype) in CI.
- **No secrets in images** — they persist in layer history; inject at runtime (env, secrets, mounted files).
- **Least privilege** — drop Linux capabilities (`--cap-drop ALL`), avoid `--privileged` , use read-only root FS (`--read-only`) where possible.
- **Resource limits** (`--memory` , `--cpus`) to contain noisy/abusive containers.
- **Keep the daemon secured** — the Docker socket is root-equivalent; never expose `dockerd` unprotected.

14. 🚀 Beyond Docker (Orchestration)

Docker runs containers on **one host**. Production at scale needs orchestration — scheduling, self-healing, scaling, rolling updates, service discovery across **many** hosts:

Tool	Role
Kubernetes (K8s)	The de-facto standard orchestrator.
Docker Swarm	Docker's built-in, simpler orchestrator.
Amazon ECS / EKS	AWS-managed container orchestration.
Nomad	HashiCorp's scheduler.

Docker images are **OCI-compliant** — the same image you build runs on Kubernetes, ECS, etc. Docker for building/local dev; an orchestrator for running fleets in production.

15. 🧠 Quick Mental Model

- **Docker = package an app + its environment into a portable container.**
- **Image** = immutable layered recipe; **container** = a running instance of it.
- Build images with a **Dockerfile** (order steps for cache; use **multi-stage** + small bases).
- Containers are **ephemeral** — persist data with **volumes**, share code in dev with **bind mounts**.
- Containers talk over **networks** (resolve by name on user-defined bridges); expose with `-p` .
- Share via **registries** (`pull` / `tag` / `push`); pin versions, don't trust `latest` in prod.
- Run multi-container stacks with **Compose**; run fleets in prod with **Kubernetes/ECS**.
- Secure it: **non-root, pinned + scanned images, no baked-in secrets, least privilege.**

 *Notes compiled as a quick reference for Docker.*